

Reconsider the N-Queens problem described in a previous exercise.

Define pseudo-code (and possibly an implementation in any high-level language) of a portfolio of programs that, given an instance of the problem (namely, a value for integer  $N > 3$ ) find a placement of  $N$  queens in a  $N \times N$  chessboard such that no queen is under attack.

Each program in the portfolio shall use a solving method inspired by a local search iterative improvement search state exploration algorithm shown in the course, and in particular:

- 1. steepest ascent/descent with restarts
- 2. first-choice hill climbing with restarts
- 3. simulated annealing.

## Funzioni comuni

Questa è una lista di funzioni usate da tutte e tre gli algoritmi

```
inCollisione(Q1, Q2):
    if Q1.row == Q2.row:
        return true

    if Q1.col == Q2.col:
        return true

    if abs(Q1.row - Q2.row) == abs(Q1.col - Q2.col):
        return true

    return false

generaVicino(stato, regina, nuovaRiga):
    nuovoStato = copia(stato)

    rimuovi regina dalla posizione corrente

    sposta regina in: (nuovaRiga, regina.col)

    aggiorna board

    return nuovoStato

costo(stato):
    conflitti = 0
```

```

    foreach coppia di regine Q1, Q2
        if inCollisione(Q1, Q2):
            conflitti++

    return conflitti

generaMosse(stato):
    mosse = []

    foreach regina Q:
        for row = 0 fino a N-1:
            if row != Q.row:
                aggiungi (Q, row) a mosse

    ritorna mosse

```

## Steepest ascent/descent

```

SteepestDescent(N, maxIter):
    stato = generaStatoCasuale(N)

    for iterazione = 1 fino a maxIter:
        if costo(stato) == 0:
            return stato

        migliore = stato
        mosse = generaMosse(stato)

        foreach (regina, nuovaRiga) in mosse:
            vicino = generaVicino(stato, regina, nuovaRiga)

            if costo(vicino) < costo(migliore):
                migliore = vicino

        if costo(migliore) < costo(stato):
            stato = migliore

```

```
    else:
        // minimo locale → restart
        stato = generaStatoCasuale(N)

return fallimento
```

## HillClimb

```
HillClimbing(N, maxIter, maxLateral):
    stato = generaStatoCasuale(N)

    iterazione = 0
    mosseLaterali = 0

    while iterazione < maxIter:
        if costo(stato) == 0:
            return stato

        mosse = generaMosse(stato)
        mescolaCasualmente(mosse)

        trovato = false

        foreach (regina, nuovaRiga) in mosse:
            vicino = generaVicino(stato, regina, nuovaRiga)

            if costo(vicino) < costo(stato):
                stato = vicino
                mosseLaterali = 0
                trovato = vero
                break
            else if costo(vicino) == costo(stato) && mosseLaterali <
maxLateral:
                stato = vicino
                mosseLaterali = mosseLaterali + 1
                trovato = true
                break
```

```
    if trovato == false:
        stato = generaStatoCasuale(N)
        mosseLateralì = 0

    iterazione = iterazione + 1

return fallimento
```

## Annealing

```
SimulatedAnnealing(N, maxIter, T0, raffreddamento):
    stato = generaStatoCasuale(N)

    T = T0
    iterazione = 0

    while iterazione < maxIter:
        if costo(stato) == 0:
            return stato

        mosse = generaMosse(stato)
        mossa = scegliCasualmente(mosse)
        (regina, nuovaRiga) = mossa
        vicino = generaVicino(stato, regina, nuovaRiga)
        deltaE = costo(vicino) - costo(stato)

        if deltaE < 0:
            stato = vicino
        else:
            prob = exp(-deltaE / T)
            if random(0, 1) < prob:
                stato = vicino

        T = aggiornaTemperatura(T, raffreddamento)
        iterazione = iterazione + 1
```

```
return fallimento
```